

Hybrid Object Detection and Segmentation with YOLO and U-Net

Ivo Kosa

I. INTRODUCTION

The problem of computer vision in robotics is a complex one. Not only does highly complex information need to be acquired from the environment, but it also needs to be reliable and often work in real time. While traditional computer vision techniques can provide robust and quick image and video analysis they lack the methods to perform complex tasks such as object recognition and localisation within the scene. Neural networks, particularly convolutional neural networks, provide a highly adaptable framework for many computer vision tasks.

The aim of this project is to produce a network, capable of both pixel-wise, multi-class semantic segmentation as well as bounding box classification and localisation. The network should also address the issue of class imbalance as some of the classes used feature very infrequently throughout the dataset. Another key point is to measure the quality of the output of the model, through metrics such as Mean Average Precision (map) and Intersection over Union (IoU).

II. DATASET

A. CamVid

The dataset used as the basis for this project is the CamVid dataset [1] [2]. It consists of 701 images, of size 960x720, split into training, testing and validation sets:

- Train: 367 Images
- Test: 233 Images
- Validation: 101 Images

For each image there is a corresponding segmented image of the same size, where each pixel value is associated with one of 32 classes. This dataset provides a good way of training a variety of computer vision models, however due to the nature of classes used and the camera as the point of view of a car, CamVid is predominantly used for use cases related to autonomous driving.

B. Dataset and Preprocessing

As CamVid does not natively provide bounding boxes, this information must be acquired during the preprocessing step. Before that can be done, first the unused classes must be filtered out. This can be easily done by setting all pixel values to black, except those matching the colour of the classes in use. Black will then be set as the background colour, or from the point of view of the network, no class present.

From this point bounding boxes can be established from the remaining data by getting the maximum and minimum x and y pixel values of each connected group of pixels. Groups

of pixels below a certain threshold will not be counted as they typically respond to artefacts in the dataset and do not accurately match the classes. Labels for the boxes can also be established through this method.

This bounding box information can then be stored in the form of COCO style json files. This is a convenient method for storing class and bounding box information compatible with a variety of libraries. These files are also able to store more information than just bounding boxes, such as encoding entire masks through methods such as Run Length Encoding (RLE).

TABLE I: Class Distributions for Test, Train, and Validation Splits

Split	Class	Bbox Count	Image Count
Test			
	car	561	220
	pedestrian	836	217
	bicyclist	196	135
	motorcycle/scooter	6	4
	truck/bus	86	70
Train			
	car	1044	332
	pedestrian	1249	326
	bicyclist	278	177
	motorcycle/scooter	8	7
	truck/bus	117	86
Val			
	car	267	89
	pedestrian	341	90
	bicyclist	78	49
	motorcycle/scooter	4	3
	truck/bus	38	28

Table I shows the distribution of the 5 classes over the training, test and validation sets. Here Bbox Count refers to the total number of bounding boxes and Image Count track for how many images that class is present in. The most obvious issue is the class imbalance concerning the motorcycle/scooter. This class has a significantly lower count than the others only appearing in 7 images in the training set. This in turn will produce poor results, when evaluating the model if left unaddressed. Similarly, the truck/bus class also has a somewhat lower count than the others, though not by a margin as small as the motorcycle/scooter class.

Another point to note is that due to the nature of the preprocessing, the background class is featured much more heavily than the others. While not an issue for bounding

box prediction, this can cause problems for the segmentation network and must also be addressed.

III. METHODOLOGY

The model architecture consists of two parts, generating the masks and identifying and classifying objects through bounding boxes. In order to address these parts a Convolutional Neural Network is needed to be able to encode and decode pixel values, with regards to the context of their environments.

A. Backbone

The first step for almost all CNNs is a backbone. This produces a general feature map, learning to find points of interest throughout the image which will help guide the detection, segmentation and classification modules of the network. In this case the backbone used is part of the resnet 34 architecture [3]. Resnet models use so-called residual connections in order to improve gradient flow throughout the model, resulting in faster convergence. This allows for a deeper network, able to learn much more complex features. In this case, the initial convolutional layers were used, as well as the encoder structure encompassing the resnet34 layers 1 to 4. Additionally, in order to speed up training, the backbone was initialised using the pre-trained weights provided by pytorch (IMAGENET1K_V1). These approximate the results of the original paper, providing a high quality feature map.

B. Image Segmentation

The image segmentation method used is based off the U-Net architecture [4]. This employs an encode-decoder method using skip connections between layers of the same shape. These skip connections are important as they preserve spatial information, which is lost through the convolutions of the encoder. After the convolutions have been processed, resulting in a feature map, the next step is the deconvolution, or decoder module of UNet. This is important for producing pixel-wise masks of an image, as this upsampling not only provides additional complexity for the model, but also recovers the lost spatial resolution. The skip connections here also help with this.

As the backbone uses 4 encoder layers the UNet detection head also uses 4 decoders in order to maintain the skip connections. These are modularised using a helper class called UpBlock, which manages the transpose convolution and skip connection. This produces the final output which gets fed to a final convolutional layer, producing an image of the correct size, with a number of channels equal to the total number of classes including the background.

C. YOLOv1

The object detection and classification is handled by the Yolo v1 architecture [5]. This is a single CNN, notable as many previous object detection methods relied on two separate networks, such as the Fast/ Faster RCNNs [6]. Yolo operates on a grid (typically 7x7) each of which predicts a given number of bounding boxes (typically 2) alongside a confidence

score, determining if the model thinks an object is present in the box. Additionally each of these grid squares predicts confidence scores that a given class is present within the cell. Given these values, the model produces an output of size:

$$[num_batches, 7, 7, 2 * 5 + num_classes]$$

These confidence scores will then be used to determine if an object is present in a given grid cell, where specifically it is and what class it is.

As pixel-wise precision is not needed, the yolo predictor head does not need any transpose convolution and is able to operate on a fully convolutional network. This consists of a series of 2D convolutions, leaky ReLU activations and batch normalisations. This then ends in a detection head, reducing the output to the aforementioned shape.

D. Loss Functions

There are two main loss functions associated with this model, one for the Yolo detection head and one for the segmentation module. The segmentation loss is comparatively much simpler than Yolo loss, in this case only using cross entropy loss between the input logits and target. It attempts to drive the values of the correct class to 1 and the incorrect classes to 0. For the Yolo loss, the standard method has been implemented, for which are 3 main aspects to the loss function: classification loss, localisation loss and confidence loss, which all train some fundamental aspect of the Yolo detector. These are all handled by mean squared error loss.

Classification loss attempts to influence the last elements of the Yolo prediction tensor, corresponding to the total number of classes. It will try to drive the prediction of the correct class in the cell to 1 and the rest to 0. Localisation loss is responsible for predicting the correct size and positioning of the bounding boxes, attempting to correctly encompass given objects. It is represented in the tensor by 4 of the 5 values multiplied by the number of bounding boxes per cell, corresponding to the x, y, w, h values of the bounding box. Finally, confidence loss is responsible for predicting if an element is present within the bounding box and is represented by the 5th tensor value for the number of bounding boxes per cell.

E. Training

The training setup used for this model is fairly standard for many machine learning models. After loading the custom training dataset using a pytorch Dataloader, the training function will run for a given number of epochs, performing back-propagation on the loss and thereby adjusting the model weights.

IV. EXPERIMENTAL SETUP

The baseline model will be trained for 50 epochs using the Adam optimiser with a learning rate of 0.001. The dataloader will shuffle the dataset and use batches of 10 in order to provide stable gradients for the learning process. The baseline model will also weight the detection and segmentation loss equally, so that the results can be more easily compared to

variations on this parameter. An early stopping mechanic has also been implemented which uses the validation training set and can be set to stop the model early if the loss does not improve after a given number of epochs, however this functionality is turned off for the baseline model. It can however be activated in order to prevent over-fitting. Finally, the no-object lambda parameter is set to 0.5 and the coordinate lambda parameter is set to 5, emulating the values used in the original paper.

These baseline parameters were chosen as they are fairly standard, given the scope of the model. This will provide a stable comparison for the modification of various parameters. This allows for easy management of the hyper-parameters, while being able to test a variety of different configurations of the model and training parameters. These can all be edited through a config.yaml file for a simple overview of all the different parameters.

Various models will then be trained and tested by modifying various aspects of the hyper-parameters as well as the training parameters such as batch size, or segmentation/ detection loss weights. The testing function will gather information from the test split by calculating mAP (mean average precision) and IoU (intersection over union), for each of the classes, as well as the overall calculation. This will allow for a large amount of data to be easily gathered leading to better understanding of the model and training parameters.

V. RESULTS

A. Baseline Model

Table II shows the results of the baseline model. The first thing to note is the lack of any accuracy regarding the motorbike class, due to the aforementioned class imbalance. This can be seen in the overall stats shown in the table as the most commonly occurring classes are by far better represented by the model than the truck and motorbike classes.

Another point to note is the overall poor performance of the yolo detector. The mAP scores show it vastly underperforms compared to the segmentation network. This may be a result of the hybrid nature of the network, which may only be able to use the backbone feature map effectively for one of the two predictor heads. Alternatively it may be as a result of the Yolo loss function, or an inconsistency with the one of the tensors used.

Despite the low score, the individual classes perform similarly to the segmentation network. It does not manage to predict any instances of the motorbike class correctly, with the majority of its correct predictions being in the pedestrian and truck/ bus classes.

Figure 1, shows a comparison of the predicted segmentation masks and bounding box classification alongside the ground truth. The results of Table II can clearly be seen with clear mask predictions for the more dominant classes of pedestrians and cars, but poor predictions for the cyclist and truck, which are much less frequent in the training set. The predicted bounding boxes are also consistent with the mAP scores,

TABLE II: Baseline Model performance metrics showing Overall and per-category mAP and IoU.

Category	mAP	IoU
Mean	0.1529	0.514
Background	–	0.9694
Car	0.1362	0.74
Pedestrian	0.2877	0.2929
Bicycle	0.0767	0.3823
Motorbike	0	0
Truck/ Bus	0.2639	0.1101

showing poor localisation quality and not picking up on some of the classes present in the image.

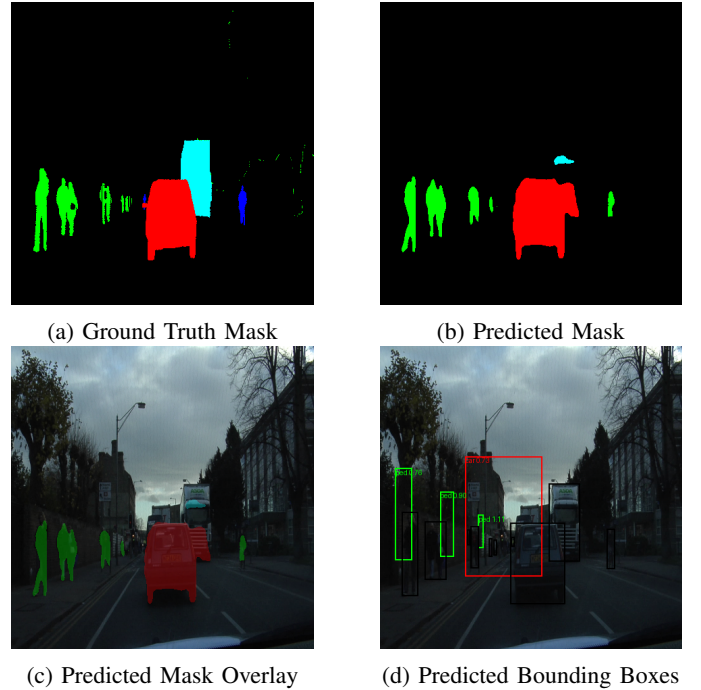


Fig. 1: Segmentation mask and bounding box prediction comparison for baseline model

B. Segmentation

This section will discuss the results for the segmentation head of the network. In order to provide the results, the model was trained without the use of the Yolo detection head, by setting the Yolo weights to 0 in config.yaml. In this case the segmentation baseline was trained on the same parameters of the baseline mentioned in section V-A without any dataset augmentations.

One of the key issues that needed addressing is the dataset imbalance addressed previously. In order to attempt to improve this aspect, a model was trained using dataset augmentation, achieved by use of the albumentations library [7]. This library introduces various image augmentations, for which a probability of occurrence can be set. For this model the dataloader class was modified in order to give a higher probability of

augmentation if an image contained an instance of the under-represented classes, in this case the motorbike and the truck/ bus class.

TABLE III: IoU comparison on baseline segmentation and augmented motorbike and truck/ bus classes

IoU	Baseline	Transforms	Transforms	Weights
Flip Prob	0	0.7	0.95	0
Rot Prob	0	0.5	0.95	0
Mean	0.5379	0.5571	0.5648	0.5559
Background	0.9717	0.9738	0.9743	0.9732
Car	0.7918	0.7923	0.7716	0.8158
Pedestrian	0.3401	0.3786	0.3479	0.3857
Bicycle	0.3962	0.4802	0.4612	0.4643
Motorbike	0	0	0	0
Truck/ Bus	0.0899	0.1007	0.1142	0.1029

Table III shows the IoU comparison between these different attempts, showing the different probabilities used. The overall results of utilising the data augmentation is positive, with both augmented models resulting in more favourable representation of the truck/ bus class. However, the motorbike class still remains as an unused class. This is likely due to the fact that since so few images contain instances of motorbikes, the augmentation is only triggered very occasionally compared to the other classes. As can be seen in the table, if enough instances are present, as with the truck/ bus class, data augmentation can be effective in increasing the overall accuracy.

Another strategy implemented was to utilise weights within the cross entropy loss. Another model was trained, without data augmentation, using weighted loss. This attempt is also shown in Table III under the 'weights' column. This attempt was somewhat successful in that the IoU of the truck/ bus class was increased over the baseline, however as with the data augmentation, the motorbike class remained unchanged.

Finally, Figure 2 visualises some of the results from the segmentation network. As can be seen the masks provide a clear boundary of many of the objects, even correctly classifying objects not shown in the ground truth data, such as the car to the left in 2a and 2b, which is shown correctly masked in the model output but not the ground truth.

2c does however also show some imperfections of the mask of the person behind the car, which is show as a four shapes of multiple different classes.

C. Detection

As previously discussed, the Yolo predictor head does not provide strong results. Training without the segmentation head, produces similar results to the hybrid model. The segmentation model alone was also tested with data augmentation, however didn't perform as well as the baseline model. Table IV shows these results. What can be seen is that the pedestrian class is comparatively much more accurate than the next most common class: car. This could be explained as the training data has more instances of pedestrians (1249) compared to cars (1044) according to Table I. The segmentation network was

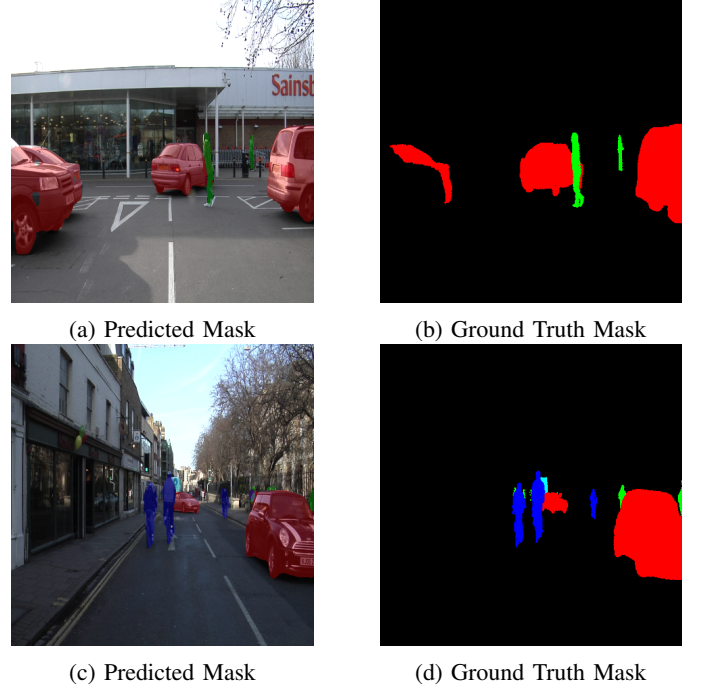


Fig. 2: Segmentation predictions taken from the weighted no-image-augmentation model

more accurate when predicting cars which had fewer instances, but were larger mask, inflating the usable training data. In this case the network does not receive more data from the size of the object and therefore the pedestrian class is more accurate.

TABLE IV: Baseline object detection compared with a model trained on augmented data

mAP	Baseline	Transforms
Flip Prob	0	0.95
Rot Prob	0	0.95
Mean	0.1565	0.1224
Car	0.1333	0.1024
Pedestrian	0.2565	0.1987
Bicycle	0.117	0.1164
Motorbike	0	0
Truck/ Bus	0.2756	0.1947

Despite this the model is significantly inaccurate, only occasionally predicting a correct object, and even then the bounding box is incorrectly localised. This would all suggest issues with the loss function.

D. Hybrid Model

The results of the full model don't differ much from the individual testing sections documented above. Figure 4 shows the results of one of the images passed through the network. The segmentation shows that some of the more predominant classes are shown with good detail, but it struggles with getting accurate information from the bus at the back. The bounding boxes correctly identify that there are cyclists and a bus in the image, but the localisation struggles with locating them.



Fig. 3: Sample predictions from the baseline Yolo network. Black boxes correspond to ground truth

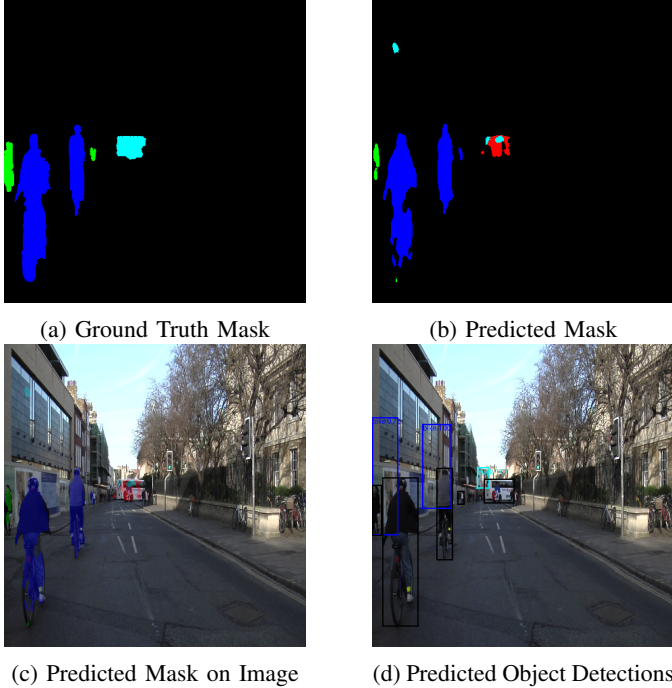


Fig. 4: Segmentation and detection results from full model

Table V shows the effects of data augmentation on the hybrid model. The augmented model was trained using 30 epochs and a flip probability of 0.5 and rotation probability of 0.2 exclusively on images containing motorbike or truck/ bus classes. The results shown are mixed, as the mAP of the model using data augmentation increased, while the IoU decreased. This is potentially an effect of the poor Yolo loss results dominating the model.

VI. DISCUSSION

The results shown in the previous section have highlighted some of the strengths and weaknesses of the model. Primarily it seems to be let down by the Yolo detector head, which produces very inaccurate predictions. Despite this, the segmentation predictor showed strong results, with good IoU scores and clear masks on visual inspection. The main other problem with the network was the poor results while working

TABLE V: Baseline model in comparison to model using data augmentation

Category	Baseline mAP	Baseline IoU	Augment mAP	Augment IoU
Mean	0.1529	0.514	0.1728	0.5002
Background	—	0.9694	—	0.9730
Car	0.1362	0.74	0.1578	0.7096
Pedestrian	0.2877	0.2929	0.3109	0.2858
Bicycle	0.0767	0.3823	0.15	0.4279
Motorbike	0	0	0	0
Truck/ Bus	0.2639	0.1101	0.2455	0.0543

with imbalanced classes. While attempts were made in order to rectify this, such as those discussed in Section V-B, the motorbike class was never correctly assigned. The primary cause of this is the lack of training data for that class. Despite some solutions being attempted, such as data augmentation and per-class loss weights, the main contribution to this issue is that there are only 7 images containing instances of this class. A better solution might have been to create additional augmented images during the pre-processing step in order to provide more stable training data. Utilising augmentations is more applicable to slightly increase the depth of training data for a whole dataset, not just for 1 or two classes. Despite this, some progress was made using augmentations, as the quality of masks in the truck/ bus class was shown to increase using these augmentations.

Further improvements could also have been made to the preprocessing step. While a threshold was placed on the size of the bounding boxes, meaning they couldn't be below a certain area, some were too close together, overlapping in some instances. Some form of non-maximal suppression could have been applied at this step.

Furthermore, while there are some advantages to the hybrid approach, making use of two different prediction heads, there are some downsides which may lead to significant performance losses. One such problem is that a shared backbone, may be forced to reconcile two incompatible networks resulting in a feature map, which limits the accuracy of both networks. In such a case opting for a more integrated network, such as Mask RCNN [8], may result in a better performing model.

As has been established one of the limiting factors is the dataset provided. It is primarily designed for image segmentation and as such much of the bounding box data is imperfect to to how it has been generated during the preprocessing step. Figure 5 shows how a large group of cars together create one singular bounding box which is difficult to automatically segment. One measure could be to define bounding box ratios and either discount or split up bounding boxes if their ratios do not match.

Another issue in this vein is that objects occluded by other objects will also provide incorrect bounding boxes (also shown in Figure 5). While there are preprocessing steps that may mitigate this, such as the ratio method mentioned before, this is currently still an issue that only manually annotated data



Fig. 5: Limitation of the bounding box data. Ground truth boxes shown in black

can provide total accuracy. This data inconsistency may be one reason why the Yolo model was unable to learn accurate bounding box information.

VII. CONCLUSION

This report has presented a hybrid Yolo-UNet decoder architecture. While limitations due to the dataset have caused issues in the representation of some classes and the accuracy of the resultant bounding boxes, there have been successes, primarily with the segmentation network. This boasts accurate masks, verified through IoU and visual inspection. Mild successes have also been seen in data augmentation, with greater IoU accuracies shown in some of the less represented classes.

Much remains to be explored with this project, such as further improvements to the pre-processing steps, creating additional training images for the low instance classes. Alternatively, other segmentation-detection networks, such as the aforementioned Mask RCNN [8]. Such a network could also make use of more advanced backbones such as the feature pyramid network [9]. This provides a series of feature maps at varying scales, allowing the network to learn different object features at multiple sizes. Alternatively, further variations on the Yolo architecture which are potentially more suited to segmentation tasks. One instance of such an architecture is YoloV11-Seg [10]. This network can provide many more functionalities than those discussed here, being able to perform instance segmentation, which defines the boundaries of specific named objects. It can also perform pose estimation on these classified objects, making it a very versatile model for a variety of computer vision tasks.

REFERENCES

- [1] Gabriel J. Brostow, Jamie Shotton, Julien Fauqueur, and Roberto Cipolla. Segmentation and recognition using structure from motion point clouds. In *ECCV (1)*, pages 44–57, 2008.
- [2] Gabriel J. Brostow, Julien Fauqueur, and Roberto Cipolla. Semantic object classes in video: A high-definition ground truth database. *Pattern Recognition Letters*, 2008.
- [3] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [4] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation, 2015.
- [5] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection, 2016.
- [6] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks, 2016.
- [7] E. Khvedchenya V. I. Iglovikov A. Buslaev, A. Parinov and A. A. Kalinin. Albumentations: fast and flexible image augmentations. *ArXiv e-prints*, 2018.
- [8] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn, 2018.
- [9] Tiancai Wang, Xiangyu Zhang, and Jian Sun. Implicit feature pyramid network for object detection, 2020.
- [10] Glenn Jocher and Jing Qiu. Ultralytics yolo11, 2024.